The core idea and the data model

Live: https://app-opal-tau-89.vercel.app - open it, then use the role switcher (top-right) to see the workpaper as a preparer, reviewer, partner, a preparer without sensitive-data clearance, firm admin, or either client. **Load demo worked file** (top bar) seeds a season's worth of events - accepted adjustments, a review note, a partner sign-off that later reopens - so there's something to look at immediately; **Reset** clears it back to just the four standing client questions. See the README for more.

The core idea: the return as an evidence graph, built on an append-only event log

A spreadsheet workpaper records a number and a cell comment. It can't answer "what changed since I checked," "who signed off on this and does it still hold," or "show me every place this document's data landed" without manual bookkeeping that rots within a season. Those aren't formatting problems - they're the wrong substrate. So this workpaper is built on two ideas instead of a grid:

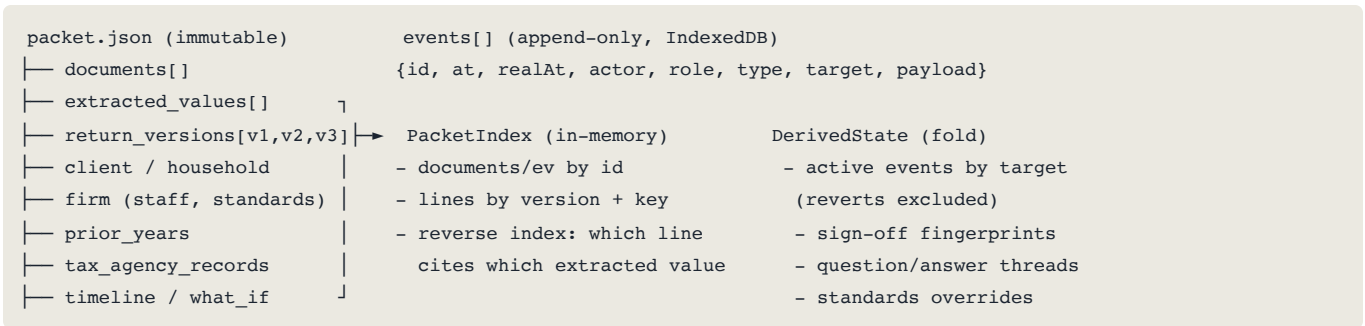
The packet is immutable source truth. `packet.json` - 74 documents, ~3,000 extracted values, three computed return versions, client answers, agency records - is never mutated. It's fetched once, indexed in memory, and treated as a historical record: what the pipeline actually produced, mistakes included.

Every human action is an event, appended to an IndexedDB log and never edited in place: verification, adjustment, review note, question, answer, sign-off, judgment, attachment, assignment, filing, a sensitive-data reveal, a standards change. Each event is `{id, at (simulated clock), realAt (wall clock), actor, role, type, target, payload}`. **Derived state is always a fold over this event list** - there is no second place that holds "the current state" to fall out of sync. The return summary, the checks list, the sign-off status, the access log: all of them are computed fresh from `events[]` every time, the same way a ledger's balance is a sum over its transactions.

One design decision, several features for free:

- **Undo** is a `revert` event pointing at the event it cancels - the fold just excludes it.
- **"Changed since I last looked"** is `events.filter(e => e.realAt > lastVisited)`.
- **Version compare** reuses the packet's own `v1 → v2 → v3` diffs, not a custom one.
- **Sign-offs reopen automatically**: a sign-off stores a fingerprint (a hash of every event touching what it covered). If a later event changes that set, the live fingerprint no longer matches and the sign-off shows as reopened - no explicit "did anything change" check anywhere, it's just arithmetic on the same fold.
- **Filing freeze, then amend on top**: freezing is itself an event; everything after it is additional events layered on the same log, not a branch or a copy.
- **A notice response is just a filtered view** of the same event log plus the packet evidence it cites - self-contained by construction, not assembled by hand when a notice arrives.

The data model



Tracing follows `return_versions[v3].lines[].sources[]` references (mostly `extracted_values` ids, some prior-year or engine-parameter pointers) back to a document and a synthetic highlight span of its `printed_content` - the `DocumentRenderer` component takes `{document, highlightEvId}` as its whole contract, so a real PDF/OCR renderer could replace the simulated paper without touching any call site.

Independent checks (`data/checks/run.ts`) are plain functions of `(PacketIndex, live adjustment state)` that recompute an expected value from raw documents/questionnaire data and compare it to the engine's line - no shared trust with `return_versions`. One check (the SEP contribution limit) recomputes live against *currently accepted* adjustments, demonstrating that checks aren't static: accepting the Schedule C fixes elsewhere lowers the SEP limit in real time.

Tax effect of any accepted adjustment is `amount × return_versions[v3].tax_effect_rates[category]` (federal and California per dollar, as the packet defines them) - or, for the HVAC capitalization question, the engine's own precomputed `what_if` alternative, since that one change cascades through depreciation, passive-loss release and AMT in ways a single rate can't capture. Every accepted adjustment also carries a **kind**: almost all of them are a *tax* effect (they move AGI and total tax); two checks - the federal and California estimated-payment corrections - are a *payments* effect instead, a dollar-for-dollar correction to payments actually made that never touches total tax. So the **workpaper total** is `v3 total tax + Σ(tax effects)` and `v3 payments − Σ(payments effects)`, with `balance due = total tax − payments` always recomputed from those two, never shifted directly - a payment correction changing total tax by mistake was an actual bug this caught. Home's top strip shows all three states per jurisdiction (engine v3 / workpaper as accepted so far / workpaper if every still-pending check were also accepted); accepting every check moves federal total tax $81,962 → $72,068 and the federal balance from $4,220 due to a $4,674 refund, and California total tax $18,874 → $12,027 with the refund growing from $8,256 to $11,603.

Roles are a fixed roster (`types/domain.ts`) matched to `firm.staff` ids, each with a `RoleId` the standards module uses to enforce the firm's starter rules live: self-review is blocked by comparing `event.actor` to `firm.assignment_at_start` , partner sign-off is required past a configurable dollar threshold, and sensitive fields (SSNs, account numbers) are masked everywhere they render - including inside the simulated source document itself - until an explicit reveal, which is itself a logged event.
